

## Edge cases (context limit) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and forcing it on purpose](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and forcing it on purpose

---

### Basic Version

Every model has a maximum amount of text it can look at in one call — the "context window." If you send more than that, the API won't quietly trim your message for you — it will refuse the whole call and give you back an error.

Your job here: make a message so long it goes past that limit on purpose, so you see the real error with your own eyes instead of just reading about it. The easiest way is repetition: a single word repeated tens of thousands of times is cheap to generate in Python and produces a huge token count fast.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

Each model has a fixed token budget for the whole request (system + user + history + the reply it's about to generate). Send a request whose token count exceeds that budget, and the API raises an error rather than silently truncating anything on your behalf — truncating your input for you would be a dangerous, silent behavior for an API to have, since it could quietly drop something important without telling you.

`huge_input = "word " * 200_000` builds a string far larger than any current model's context window in a single line. Wrap the API call in `try: ... except openai.BadRequestError as e:` — a context-length problem surfaces as a `BadRequestError` (HTTP 400), not a special "context" exception of its own, so catching that specific type (not a bare `except:` ) is what actually shows you which real exception class this is. Print `type(e)` and `str(e)` — the message text usually names the token counts involved directly, which is worth reading once for real.

**Difference between Basic and Intermediate:** Basic reacts to the limit with a bare `except Exception`. Intermediate catches `openai.BadRequestError` specifically — the same "catch the specific error you know how to handle" rule from Doc01.



[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
build a huge piece of text (a word repeated many, many times)

try:
    send it to the model
except (something went wrong):
    print what kind of error it was
    print the error's message
```

Here's almost the whole thing:

```
# context_limit_practice.py
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print(type(e).__name__, str(e)[:200])
```

**Expected output:** something naming `BadRequestError` and a message mentioning the model's maximum context length and roughly how many tokens your message actually was.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
import openai
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(type(e).__name__)
    print(str(e))
```

Run it — then compare catching the *specific* error type against the bare `Exception` from Basic. Notice `openai.BadRequestError` is what you'd actually write in real code, since a bare `except` would also silently swallow a genuine bug in your own request-building code, not just this one expected failure. Then compare against the [Solution](#).

**Difference between Basic and Intermediate:** both discover the limit reactively — send first, catch the failure after. Intermediate catches the specific `BadRequestError` type instead of a bare `Exception`, documenting exactly what failure this code is prepared for.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)